



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-27144 Go cmd/compile miscompilation can cause runtime memory corruption

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-09
Version	1.1
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-27144 Go cmd/compile miscompilation can cause runtime memory corruption - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-09 | TLP:CLEAR | Version: 1.1 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
8. Mitigation and Workarounds
9. Detection Engineering
10. Threat Hunting
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. References

CVE-2026-27144 Go cmd/compile miscompilation can cause runtime memory corruption

1. Executive Summary

CVE-2026-27144 is a high-severity Go toolchain flaw in `cmd/compile` where a no-op interface conversion (`CONVNOP`) can interfere with overlap checking for array-copy style memory moves. Upstream Go issue tracking, the CNA CVE record, and downstream vulnerability databases align on the core problem: the compiler can incorrectly decide that a move is non-overlapping and emit code that allows runtime memory corruption.

This is not best understood as a classic remotely triggerable service bug. The primary exposure is to organizations building or shipping software with affected Go toolchains, especially where security-sensitive code paths, parsers, agents, or multi-tenant services are compiled with vulnerable releases. The strongest immediate action is to upgrade build environments and rebuild affected binaries with fixed Go versions `1.25.9`, `1.26.2`, or later maintained releases.

2. Vulnerability Metadata

Field	Value
CVE	<code>CVE-2026-27144</code>
Go advisory ID	<code>GO-2026-4867</code>
Product / component	Go toolchain <code>cmd/compile</code>
Vulnerability class	compiler miscompilation leading to potential runtime memory corruption
Primary weakness framing	<code>CWE-440: Expected Behavior Violation</code>
CVSS v3.1	<code>7.1 / HIGH</code>
CVSS vector	<code>CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:H</code>
Affected version branches	before <code>Go 1.25.9</code> , and <code>Go 1.26.0</code> through before <code>Go 1.26.2</code>
Fixed versions	<code>Go 1.25.9</code> and <code>Go 1.26.2</code>
CNA publication date	<code>2026-04-08</code>
Upstream tracking	Go issue <code>#78371</code>
Credit	Jakub Ciolek

3. Source Review & Confidence

Source	Contribution	Confidence
Reviewed source	MSRC lists <code>CVE-2026-27144</code> and was the originating feed item for this intake review.	Useful as the intake and publication reference, but not the richest technical source.
Reviewed source	The Go CNA CVE record describes a <code>cmd/compile</code> miscompilation issue, assigns <code>CWE-440</code> , marks affected versions before	Primary structured vulnerability source.

Reviewed source	Go's vulnerability entry ties `GO-2026-4867` to `CVE-2026-27144`, `cmd/compile`, and the same compiler-memory-corruption	Primary vendor-maintained ecosystem source.
Reviewed source	Upstream Go issue states that the compiler failed to unwrap pointers inside a no-op interface conversion, causing incorr	Strong technical source for root-cause and patch-lineage context.
Reviewed source	Go announced security releases `1.26.2` and `1.25.9`, confirming the fixed maintenance versions for this issue.	Primary release-validation source.
Primary source	NVD records public publication and mirrors the Go Project as the vendor advisory source.	Useful for publication status and downstream severity/corroboration, but less detailed than Go-nativ
Reviewed source	Red Hat tracks the vulnerability downstream as a Go security issue and classifies it with medium severity in its ecosyst	Good downstream product-tracking corroboration.
Reviewed source	Ubuntu tracks the CVE as a Go-related issue, providing additional downstream distro awareness.	Useful corroboration for ecosystem tracking, though sparse on technical detail.

Confidence is high on the core finding because multiple authoritative sources agree on the vulnerability mechanics and remediation path: Confidence is lower on exploitation specifics because the reviewed public material does not document in-the-wild exploitation, public proof-of-concept weaponization, or CISA KEV inclusion. The strongest supportable claim is that this is a real compiler correctness and integrity issue that can cause dangerous runtime behavior in binaries built with vulnerable toolchains.

4. Vulnerability Details

The vulnerable behavior sits in the Go compiler rather than in application logic. According to the upstream issue and CNA description, the compiler should unwrap pointers that participate in memory-move operations when determining whether the source and destination overlap. A no-op interface conversion (`CONVNOP`) prevented the compiler from performing that analysis correctly.

As a result, the compiler could treat an overlapping move as safe and non-overlapping, which is precisely the kind of mistaken optimization or code-generation decision that can lead to memory corruption at runtime. The impact is therefore delayed: developers compile code successfully, but the resulting program may execute unsafe memory operations under the right conditions.

Key operational implications:

- vulnerable source code may compile without warning
- the faulty behavior manifests in the produced binary, not only at build time
- the risk is integrity and availability oriented, with potential crashes, corrupted state, or other unsafe runtime outcomes
- environments that trust compiler correctness for security-sensitive components should treat this as a supply-chain and build-integrity patching problem

5. Attack Scenarios

1. A development team builds a security-sensitive Go service with a vulnerable toolchain. Specific code patterns involving interface conversions and array-copy style moves compile into unsafe runtime behavior, causing crashes or state corruption in production.

2. A software supplier, CI farm, or package maintainer distributes binaries built with affected Go versions. Downstream customers inherit the defect even if their own systems never run the vulnerable compiler directly.
3. An attacker who can influence what code is compiled in a shared build environment may attempt to craft source patterns that trigger the compiler flaw, seeking denial of service or subtle integrity failures in the resulting binary.
4. Internal agents, gateways, or data-processing services written in Go and built with vulnerable versions may experience memory-safety failures in edge cases that are hard to diagnose because the defect originates in compilation rather than obvious application logic.

The public evidence reviewed during this task does not support a claim of turnkey remote exploitation against arbitrary internet-facing services. Exposure depends on whether the target software was built with an affected toolchain and whether reachable runtime paths exercise the miscompiled logic.

6. Threat Landscape & Exploitation Status

- Public disclosure exists across the Go CNA CVE record, Go issue tracker, Go security release announcement, NVD, MSRC, and downstream distro/vendor trackers.
- CISA ADP enrichment records `Exploitation: none`, `Automatable: no`, and `Technical Impact: partial`, which is consistent with a serious but not broadly commoditized issue.
- No evidence was collected during this review showing CISA KEV inclusion or confirmed in-the-wild exploitation.
- The vulnerability is best framed as a high-severity build and software-supply-chain risk: organizations may unknowingly ship flawed binaries if their Go maintenance releases lag.
- The issue carried security and release-blocker labels upstream, indicating the Go maintainers treated it as important for supported release branches.

7. Affected Products and Exposure

The most defensible exposure statement is that affected products are binaries compiled with vulnerable Go toolchains before `1.25.9` or in the vulnerable `1.26.0`-`1.26.1` range when the compiled code exercises the affected memory-move logic.

Higher-risk patterns:

- Organizations with self-hosted Go build pipelines, CI/CD runners, or package-build farms that lagged security releases.
- Vendors shipping security agents, network services, parsers, or platform components built in Go where crashes or silent corruption would be operationally significant.
- Multi-tenant or safety-sensitive services where compiler-induced corruption could degrade service integrity or availability.
- Teams consuming prebuilt binaries from suppliers who may have compiled with vulnerable toolchains.

Lower-risk patterns:

- Environments already rebuilt using `Go 1.25.9`, `Go 1.26.2`, or later maintained releases.
- Workloads that do not use affected code-generation patterns, though the reviewed public material does not provide a simple static signature for proving non-exposure without rebuild and validation.
- Organizations that can quickly inventory build provenance and attest that production binaries were produced with fixed Go versions.

8. Mitigation and Workarounds

Priority actions:

5. Upgrade all supported Go build environments to `1.25.9`, `1.26.2`, or later maintained releases.
6. Rebuild and redeploy production binaries, libraries, containers, and internal tools that may have been compiled with affected Go versions.
7. Review software bills of materials, build logs, CI metadata, and container provenance to identify binaries built during the vulnerable window.

8. Prioritize patching for security-sensitive Go services, parsers, agents, authentication components, and software distributed to customers.
9. If immediate rebuilds are not possible, identify critical binaries compiled with vulnerable Go versions and apply compensating monitoring for crashes, abnormal exits, or unexplained state corruption while patching proceeds.

There is no strong public evidence for a practical runtime-only workaround that removes the defect without rebuilding. The safest path is patched compiler adoption plus fresh builds.

9. Detection Engineering

Detection is primarily about version and build-provenance discovery rather than IOC matching:

Signal	Telemetry Source	Analytic Goal
Inventory Go versions present on developer workstations, CI runners, build containers, release automation, and package-build infrastructure.	Asset inventory, SBOM, package management, CI/CD telemetry	Find vulnerable builds or packages
Search build logs, artifact metadata, SBOMs, and container labels for `go1.25.8` or earlier on the 1.25 branch and `go1.26.0`-`go1.26.1` on the 1.26 branch.	Asset inventory, SBOM, package management, CI/CD telemetry	Find vulnerable builds or packages
Flag production binaries whose build provenance cannot rule out vulnerable Go compiler versions.	Asset inventory, SBOM, package management, CI/CD telemetry	Find vulnerable builds or packages
Monitor Go services built during the exposure window for unexplained crashes, panics, memory-corruption symptoms, or integrity anomalies after deployment.	Application logs, service metrics, EDR / crash telemetry	Spot exploitation symptoms or instability
Review third-party Go software intake to determine whether upstream vendors have issued rebuild or patch notices.	Asset inventory, SBOM, package management, CI/CD telemetry	Find vulnerable builds or packages

9.1 Detection Query

```
// Build and package provenance pivots for CVE-2026-27144 Go cmd/compile miscompilation can cause runtime memory corruption
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ProcessCommandLine has_any ("go version", "go env", "pip show", "pip list", "syft", "grype", "sbom")
| project Timestamp, DeviceName, InitiatingProcessAccountName, FileName, ProcessCommandLine
| order by Timestamp desc
```

9.2 Exposure / Triage Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine has_any ("panic", "fatal error")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

10. Threat Hunting

Suggested hunt questions:

- Which production or customer-distributed binaries were built with Go versions earlier than `1.25.9` or in the `1.26.0`-`1.26.1` range?
- Which CI runners, container images, and build templates still embed vulnerable Go releases?
- Do any recent incidents involving Go services show unexplained crashes or corruption after builds produced during the vulnerable period?
- Which third-party Go-based products in the environment require vendor confirmation that they were rebuilt with fixed toolchains?
- Do artifact repositories, release pipelines, or SBOM stores preserve enough provenance to distinguish vulnerable and rebuilt outputs?

10.1 Hunt Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine
has_any ("panic", "fatal error")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

11. MITRE ATT&CK Mapping

No ATT&CK technique is assigned with high confidence for this intake review. The public evidence supports a compiler miscompilation and software-supply-chain integrity issue, but it does not document a concrete adversary procedure or intrusion chain that would justify a precise ATT&CK mapping without over-claiming.

12. Validation / Lab Testing

This intake review validated the report against public advisory, upstream issue, and release-tracking material rather than performing a fresh proof-of-concept reproduction. Analyst validation steps completed:

- Reviewed the MSRC Update Guide listing that triggered the intake.
- Reviewed the Go CNA CVE record for description, affected versions, CWE, CVSS, and references.
- Reviewed the upstream Go security issue `#78371` and associated backport/fix references.
- Reviewed Go security release announcement coverage showing the fix shipped in `1.26.2` and `1.25.9`.
- Reviewed package-vulnerability, NVD, and downstream vendor-tracker corroboration for ecosystem impact.

No Lost Boys Cyber exploit reproduction or compiler-trigger validation was performed as part of this draft review.

13. Recommended Actions Summary

Priority	Owner	Action
Critical	Platform / Application Owners	Upgrade Go toolchains to `1.25.9`, `1.26.2`, or later maintained releases.
Critical	Platform / Application Owners	Rebuild and redeploy binaries compiled during the vulnerable period; patching the compiler alone is not enough if old artifacts remain in circulation.
High	Security / Platform Owners	Focus first on customer-delivered software, exposed services, internal agents, and other high-trust Go binaries.
High	Third-Party Risk / Asset Management	Improve build provenance and SBOM visibility so vulnerable compiler lineage can be identified quickly in future supply-

		chain events.
High	Third-Party Risk / Asset Management	Track vendor advisories for third-party Go software and require confirmation that fixed toolchains were used for rebuilt releases.

14. References

10. Microsoft Security Response Center, `CVE-2026-27144`
11. MITRE CVE record for `CVE-2026-27144`
12. Go vulnerability database entry `GO-2026-4867`
13. Go issue tracker `#78371`
14. Go security release announcement for `1.26.2` and `1.25.9`
15. NVD entry for `CVE-2026-27144`
16. Red Hat Bugzilla tracking for `CVE-2026-27144`
17. Ubuntu CVE tracker for `CVE-2026-27144`